

# Taller de Lógica: Jerarquía y Creación de Procesos

Malak Sanchez

Monitorías de Sistemas Operativos

19 de marzo de 2026

## Introducción

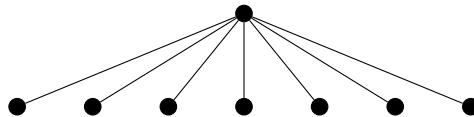
Este taller se enfoca puramente en la lógica de creación y herencia de procesos mediante la llamada al sistema `fork()`. No se requiere sincronización ni comunicación avanzada; el objetivo es que el estudiante domine la visualización de estructuras complejas y el comportamiento de los procesos en bifurcación.

## Bloque 1: De Diagramas a Código

*Desarrolle el código necesario para replicar exactamente las siguientes estructuras de jerarquía.*

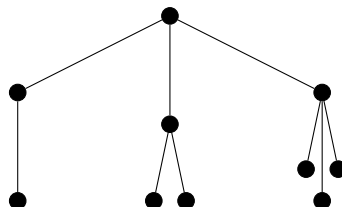
### Ejercicio 1

Un proceso padre que genera exactamente 7 procesos hijos de forma paralela.



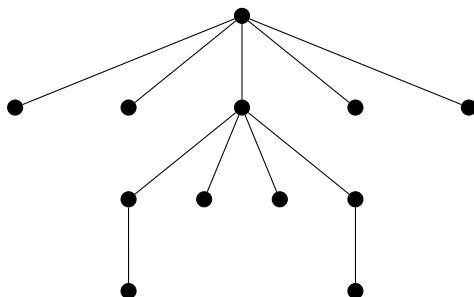
### Ejercicio 2

Un proceso raíz con 3 hijos: el primero tiene 1 hijo, el segundo tiene 2 hijos y el tercero tiene 3 hijos.



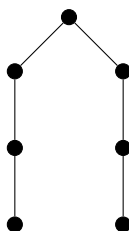
### Ejercicio 3

Replique la jerarquía de la imagen: Un proceso raíz con 5 hijos, donde el tercer hijo crea a su vez 4 hijos. Los extremos de esta última generación (el primero y el cuarto) crean un proceso final cada uno.



### Ejercicio 4

Un padre con 2 hijos. Cada hijo crea 1 nieto, y cada nieto crea 1 bisnieto.



### Ejercicio 5

Desarrolle una función recursiva `void expandir(int n)` que genere un árbol binario perfecto de profundidad 3. Dibuje el árbol resultante e indique el número total de procesos.

## Bloque 2: De Código a Diagramas

*Analice los siguientes fragmentos de código y dibuje el árbol de procesos resultante. Indique cuántos procesos totales se crearon (incluyendo al padre original).*

### Ejercicio 6

```
1  int n = 2;
2  for(int i=0; i < n; i++) {
3      fork();
4      fork();
5  }
```

### Ejercicio 7

```
1  if(fork() || fork()) {
2      fork();
3  }
```

### Ejercicio 8

```
1  if(fork() && fork()) {
2      fork();
3  }
```

### Ejercicio 9

```
1  pid_t p;
2  p = fork();
3  if(p != 0) {
4      fork();
5  }
6  fork();
```

## Ejercicio 10

Analice qué sucede en este código y dibuje la jerarquía generada.

```
1 void generar(int n) {  
2     if(n <= 0) return;  
3     if(fork() == 0) {  
4         generar(n-1);  
5     }  
6     fork();  
7 }  
8  
9 int main() {  
10     generar(2);  
11     return 0;  
12 }
```

## Ejercicio 11

```
1 pid_t p;  
2 p = fork();  
3 if(p = 0) {  
4     fork();  
5 }
```